

Web Technology

Class Lecture
+
Lab Session
9 Hours

Unit- Unit IV: Client Side Scripting with JavaScript

4.1. Structure of JavaScript Program

4.2. Variables and Data Types; Statements: Expression, Keyword, Block; Operators

4.3. Flow Controls, Looping

4.4. Functions; Popup Boxes: Alert, Confirm, Prompt

4.5. Objects and properties; Constructors

4.6. Arrays

4.7. Built-in Objects: Window, String, Number, Boolean, Date, Math, RegExp, Form, DOM

4.8. User Defined Objects; Event Handling and Form Validation

4.9. Error Handling (Try/Catch),

4.10. Handling Cookies

4.11. jQuery Syntax; jQuery Selectors (Element, Id, Class), jQuery Events (Mouse, Keyboard, Form, Document/Window) and jQuery Effects (Hide/Show, Fade, Slide, Animate, Stop, Callback, Chaining)

4.12. Introduction to JSON (Syntax, Data types, Parsing JSON)

❑ What is Java Script

- JavaScript is a lightweight, interpreted programming language.
- It is designed for creating network-centric applications.
- It is complimentary to and integrated with Java.
- JavaScript is very easy to implement because it is integrated with HTML.
- It is open and cross-platform.

❑ Why to Learn Javascript

- Javascript is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain.
- some of the key advantages of learning Javascript:

The ECMA-262 Specification defined a standard version of the core JavaScript language.

- JavaScript is a lightweight, interpreted programming language.
- Designed for creating network-centric applications.
- Complementary to and integrated with Java.
- Complementary to and integrated with HTML.
- Open and cross-platform

1. Javascript is the most popular programming language in the world and that makes it a programmer's great choice. Once you learnt Javascript, it helps you developing great front-end as well as back-end softwares using different Javascript based frameworks like **jQuery, Node.JS etc.**
2. Javascript is everywhere, it comes installed on every modern web browser and so to learn Javascript you really do not need any special environment setup. For **example Chrome, Mozilla Firefox , Safari** and every browser you know as of today, supports Javascript.
3. Javascript helps you create really beautiful and crazy fast websites. You can develop your website with a console like look and feel and give your users the best Graphical User Experience.
4. JavaScript usage has now extended to mobile app development, desktop app development, and game development. This opens many opportunities for you as Javascript Programmer.
5. Due to high demand, there is tons of job growth and high pay for those who know JavaScript. You can navigate over to different job sites to see what having JavaScript skills looks like in the job market.
6. Great thing about Javascript is that you will find tons of frameworks and Libraries already developed which can be used directly in your software development to reduce your time to market.

There are many useful Javascript frameworks and libraries available:

Angular React jQuery Vue.js Ext.js
Ember.js Meteor Mithril Node.js
Polymer Aureli Backbone.js

❑ Applications of Javascript Programming

- As mentioned before, Javascript is one of the most widely used programming languages (Front-end as well as Back-end). It has its presence in almost every area of software development. I'm going to list a few of them here:
- Client side validation - This is really important to verify any user input before submitting it to the server and Javascript plays an important role in validating those inputs at front-end itself.
- Manipulating HTML Pages - Javascript helps in manipulating HTML page on the fly. This helps in adding and deleting any HTML tag very easily using javascript and modify your HTML to change its look and feel based on different devices and requirements.
- User Notifications - You can use Javascript to raise dynamic pop-ups on the webpages to give different types of notifications to your website visitors.

- Back-end Data Loading - Javascript provides Ajax library which helps in loading back-end data while you are doing some other processing. This really gives an amazing experience to your website visitors.
- Presentations - JavaScript also provides the facility of creating presentations which gives website look and feel. JavaScript provides RevealJS and BespokeJS libraries to build a web-based slide presentations.
- Server Applications - Node JS is built on Chrome's Javascript runtime for building fast and scalable network applications. This is an event based library which helps in developing very sophisticated server applications including Web Servers.

❑ Client-Side JavaScript

- Client-side JavaScript is the most common form of the language. The script should be included in or referenced by an HTML document for the code to be interpreted by the browser.
- It means that a web page need not be a static HTML, but can include programs that interact with the user, control the browser, and dynamically create HTML content.
- The JavaScript client-side mechanism provides many advantages over traditional CGI server-side scripts. For example, you might use JavaScript to check if the user has entered a valid e-mail address in a form field.
- The JavaScript code is executed when the user submits the form, and only if all the entries are valid, they would be submitted to the Web Server.
- JavaScript can be used to trap user-initiated events such as button clicks, link navigation, and other actions that the user initiates explicitly or implicitly.

❑ Advantages of JavaScript

The merits of using JavaScript are –

- Less server interaction – You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- Immediate feedback to the visitors – They don't have to wait for a page reload to see if they have forgotten to enter something.
- Increased interactivity – You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- Richer interfaces – You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.

❑ Limitations of JavaScript

- We cannot treat JavaScript as a full-fledged programming language. It lacks the following important features –
- Client-side JavaScript does not allow the reading or writing of files. This has been kept for security reason.
- JavaScript cannot be used for networking applications because there is no such support available.
- JavaScript doesn't have any multi-threading or multiprocessor capabilities.

❑ JavaScript Development Tools

- One of major strengths of JavaScript is that it does not require expensive development tools. You can start with a simple text editor such as Notepad. Since it is an interpreted language inside the context of a web browser, you don't even need to buy a compiler.
- To make our life simpler, various vendors have come up with very nice JavaScript editing tools. Some of them are listed here –
- Microsoft FrontPage – Microsoft has developed a popular HTML editor called FrontPage. FrontPage also provides web developers with a number of JavaScript tools to assist in the creation of interactive websites.
- Macromedia Dreamweaver MX – Macromedia Dreamweaver MX is a very popular HTML and JavaScript editor in the professional web development crowd. It provides several handy prebuilt JavaScript components, integrates well with databases, and conforms to new standards such as XHTML and XML.
- Macromedia HomeSite 5 – HomeSite 5 is a well-liked HTML and JavaScript editor from Macromedia that can be used to manage personal websites effectively.

❑ JavaScript - Syntax

- JavaScript can be implemented using JavaScript statements that are placed within the `<script>... </script>` HTML tags in a web page.
- You can place the `<script>` tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the `<head>` tags.
- The `<script>` tag alerts the browser program to start interpreting all the text between these tags as a script. A simple syntax of your JavaScript will appear as follows

```
        <script ...>
        JavaScript code
        </script>.
```

- The script tag takes two important attributes –
 - **Language** – This attribute specifies what scripting language you are using. Typically, its value will be javascript. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.
 - **Type** – This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".

So your JavaScript segment will look like –

```
<script language = "javascript" type = "text/javascript">
    JavaScript code
</script>
```

```
<html>
  <body>
    <script language = "javascript" type = "text/javascript">
      <!--
        document.write("Hello World!")
      //-->
    </script>
  </body>
</html>
```

Some fact about Javascript

Whitespace and Line Breaks

JavaScript ignores spaces, tabs, and newlines that appear in JavaScript programs. You can use spaces, tabs, and newlines freely in your program and you are free to format and indent your programs in a neat and consistent way that makes the code easy to read and understand.

Semicolons are Optional

Simple statements in JavaScript are generally followed by a semicolon character, just as they are in C, C++, and Java. JavaScript, however, allows you to omit this semicolon if each of your statements are placed on a separate line. For example, the following code could be written without semicolons.

```
<script language = "javascript" type = "text/javascript">
  <!--
    var1 = 10
    var2 = 20
  //-->
</script>
```

But when formatted in a single line as follows, you must use semicolons –

```
<script language = "javascript" type = "text/javascript">
  <!--
    var1 = 10; var2 = 20;
  //-->
</script>
```

Note – It is a good programming practice to use semicolons.

Case Sensitivity

JavaScript is a case-sensitive language. This means that the language keywords, variables, function names, and any other identifiers must always be typed with a consistent capitalization of letters.

So the identifiers Time and TIME will convey different meanings in JavaScript.

NOTE – Care should be taken while writing variable and function names in JavaScript.

Comments in JavaScript

- JavaScript supports both C-style and C++-style comments, Thus –
- Any text between a // and the end of a line is treated as a comment and is ignored by JavaScript.
- Any text between the characters /* and */ is treated as a comment. This may span multiple lines.
- JavaScript also recognizes the HTML comment opening sequence <!--. JavaScript treats this as a single-line comment, just as it does the // comment.
- The HTML comment closing sequence --> is not recognized by JavaScript so it should be written as //-->.

Example

The following example shows how to use comments in JavaScript.

```
<script language = "javascript" type =  
"text/javascript">  
    <!--  
        // This is a comment. It is similar to comments  
        in C++  
  
        /*  
        * This is a multi-line comment in JavaScript  
        * It is very similar to comments in C  
        Programming  
        */  
    //-->  
</script>
```

JavaScript in Internet Explorer

Here are simple steps to turn on or turn off JavaScript in your Internet Explorer –

- Follow Tools → Internet Options from the menu.
- Select Security tab from the dialog box.
- Click the Custom Level button.
- Scroll down till you find Scripting option.
- Select Enable radio button under Active scripting.
- Finally click OK and come out

JavaScript in Firefox

- Here are the steps to turn on or turn off JavaScript in Firefox –
- Open a new tab → type about: config in the address bar.
- Then you will find the warning dialog. Select I'll be careful, I promise!
- Then you will find the list of configure options in the browser.
- In the search bar, type javascript.enabled.
- There you will find the option to enable or disable javascript by right-clicking on the value of that option → select toggle.

JavaScript in Chrome

Here are the steps to turn on or turn off JavaScript in Chrome –

Click the Chrome menu at the top right hand corner of your browser.

Select Settings.

Click Show advanced settings at the end of the page.

Under the Privacy section, click the Content settings button.

In the "Javascript" section, select "Do not allow any site to run JavaScript" or "Allow all sites to run JavaScript (recommended)".

JavaScript in Opera

Here are the steps to turn on or turn off JavaScript in Opera –

Follow Tools → Preferences from the menu.

Select Advanced option from the dialog box.

Select Content from the listed items.

Select Enable JavaScript checkbox.

Finally click OK and come out.

❑ Warning for Non-JavaScript Browsers

If you have to do something important using JavaScript, then you can display a warning message to the user using `<noscript>` tags.

You can add a noscript block immediately after the script block as follows –

```
<html>
  <body>
    <script language = "javascript" type = "text/javascript">
      <!--
        document.write("Hello World!")
      //-->
    </script>

    <noscript>
      Sorry...JavaScript is needed to go ahead.
    </noscript>
  </body>
</html>
```

Now, if the user's browser does not support JavaScript or JavaScript is not enabled, then the message from `</noscript>` will be displayed on the screen.

❏ JavaScript - Placement in HTML File

There is a flexibility given to include JavaScript code anywhere in an HTML document. However the most preferred ways to include JavaScript in an HTML file are as follows –

- Script in <head>...</head> section.
- Script in <body>...</body> section.
- Script in <body>...</body> and <head>...</head> sections.
- Script in an external file and then include in <head>...</head> section.

In the following section, we will see how we can place JavaScript in an HTML file in different ways.

JavaScript in <head>...</head> section

- If you want to have a script run on some event, such as when a user clicks somewhere, then you will place that script in the head as follows –

```
<html>
  <head>
    <script type = "text/javascript">
      <!--
        function sayHello() {
          alert("Hello World")
        }
      //-->
    </script>
  </head>

  <body>
</body> <input type = "button" onclick =
"sayHello()" value = "Say Hello
</body>
</html>
```

❑ JavaScript in <body>...</body> section

If you need a script to run as the page loads so that the script generates content in the page, then the script goes in the <body> portion of the document. In this case, you would not have any function defined using JavaScript. Take a look at the following code.

```
<html>
  <head>
  </head>

  <body>
    <script type = "text/javascript">
      <!--
        document.write("Hello World")
      //-->
    </script>

    <p>This is web page body </p>
  </body>
</html>
```

❑ JavaScript in <body> and <head> Sections

You can put your JavaScript code in <head> and <body> section altogether as follows –

```
<html>
  <head>
    <script type = "text/javascript">
      <!--
        function sayHello() {
          alert("Hello World")
        }
      //-->
    </script>
  </head>

  <body>
    <script type = "text/javascript">
      <!--
        document.write("Hello World")
      //-->
    </script>

    <input type = "button" onclick = "sayHello()" value = "Say Hello" />
  </body>
</html>
```

❑ JavaScript in External File

- As you begin to work more extensively with JavaScript, you will be likely to find that there are cases where you are reusing identical JavaScript code on multiple pages of a site.
- You are not restricted to be maintaining identical code in multiple HTML files. The script tag provides a mechanism to allow you to store JavaScript in an external file and then include it into your HTML files.
- Here is an example to show how you can include an external JavaScript file in your HTML code using script tag and its src attribute.

```
<html>
  <head>
    <script type = "text/javascript" src = "filename.js" ></script>
  </head>

  <body>
    .....
  </body>
</html>
```

- To use JavaScript from an external file source, you need to write all your JavaScript source code in a simple text file with the extension ".js" and then include that file as shown above.
- For example, you can keep the following content in filename.js file and then you can use sayHello function in your HTML file after including the filename.js file.
- ```
function sayHello() {
 alert("Hello World")
}
```

## ❏ JavaScript Output

➤ JavaScript can "display" data in different ways:

1. Writing into an HTML element, using `innerHTML`.
2. Writing into the HTML output using `document.write()`.
3. Writing into an alert box, using `window.alert()`.
4. Writing into the browser console, using `console.log()`.

## 4.2. Variables and Data Types; Statements: Expression, Keyword, Block; Operators

### ❑ JavaScript - Variables

#### ❑ JavaScript Datatypes

- One of the most fundamental characteristics of a programming language is the set of data types it supports. These are the type of values that can be represented and manipulated in a programming language.
- JavaScript allows you to work with three primitive data types –
  - Numbers, eg. 123, 120.50 etc.
  - Strings of text e.g. "This text string" etc.
  - Boolean e.g. true or false.
- JavaScript also defines two trivial data types, null and undefined, each of which defines only a single value. In addition to these primitive data types, JavaScript supports a composite data type known as object. We will cover objects in detail in a separate chapter.

Note – JavaScript does not make a distinction between integer values and floating-point values. All numbers in JavaScript are represented as floating-point values. JavaScript represents numbers using the 64-bit floating-point format defined by the IEEE 754 standard.

## ❏ 4 Ways to Declare a JavaScript Variable:

- Using var
- Using let
- Using const
- Using nothing

## ❑ JavaScript Variables

- Like many other programming languages, JavaScript has variables. Variables can be thought of as named containers. You can place data into these containers and then refer to the data simply by naming the container.
- Before you use a variable in a JavaScript program, you must declare it. Variables are declared with the var keyword as follows.

```
<script type = "text/javascript">
 <!--
 var money;
 var name;
 //-->
</script>
```

```
<script type =
"text/javascript">
 <!--
 var money, name;
 //-->
</script>
```

Storing a value in a variable is called variable initialization. You can do variable initialization at the time of variable creation or at a later point in time when you need that variable.

For instance, you might create a variable named money and assign the value 2000.50 to it later. For another variable, you can assign a value at the time of initialization as follows.

```
<script type = "text/javascript">
 <!--
 var name = "Ali";
 var money;
 money = 2000.50;
 //-->
</script>
```

Note – Use the var keyword only for declaration or initialization, once for the life of any variable name in a document. You should not re-declare same variable twice.

JavaScript is untyped language. This means that a JavaScript variable can hold a value of any data type. Unlike many other languages, you don't have to tell JavaScript during variable declaration what type of value the variable will hold. The value type of a variable can change during the execution of a program and JavaScript takes care of it automatically.

## ❑ JavaScript Variable Scope

- The scope of a variable is the region of your program in which it is defined. JavaScript variables have only two scopes.
- Global Variables – A global variable has global scope which means it can be defined anywhere in your JavaScript code.
- Local Variables – A local variable will be visible only within a function where it is defined. Function parameters are always local to that function.
- Within the body of a function, a local variable takes precedence over a global variable with the same name. If you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable. Take a look into the following example.

```
<html>
 <body onload = checkscope();>
 <script type = "text/javascript">
 <!--
 var myVar = "global"; // Declare a global
variable
 function checkscope() {
 var myVar = "local"; // Declare a local
variable
 document.write(myVar);
 }
 //-->
 </script>
 </body>
</html>
```

## ❑ JavaScript Variable Names

- While naming your variables in JavaScript, keep the following rules in mind.
- You should not use any of the JavaScript reserved keywords as a variable name. These keywords are mentioned in the next section. For example, break or boolean variable names are not valid.
- JavaScript variable names should not start with a numeral (0-9). They must begin with a letter or an underscore character. For example, 123test is an invalid variable name but \_123test is a valid one.
- JavaScript variable names are case-sensitive. For example, Name and name are two different variables.

## ❑ JavaScript Reserved Words

A list of all the reserved words in JavaScript are given in the following table. They cannot be used as JavaScript variables, functions, methods, loop labels, or any object names

abstract	else	instanceof	switch
boolean	enum	int	synchronized
break	export	interface	this
byte	extends	long	throw
case	false	native	throws
catch	final	new	transient
char	finally	null	true
class	float	package	try
const	for	private	typeof
continue	function	protected	var
debugger	goto	public	void
default	if	return	volatile
delete	implements		short while
do	import	static	with
double	in	super	

## ❏ JavaScript - Operators

### What is an Operator?

➤ Let us take a simple expression  $4 + 5$  is equal to 9. Here 4 and 5 are called operands and '+' is called the operator. JavaScript supports the following types of operators.

1. Arithmetic Operators
2. Comparison Operators
3. Logical (or Relational) Operators
4. Assignment Operators
5. Conditional (or ternary) Operators

### 1. Arithmetic Operators

JavaScript supports the following arithmetic operators –

Assume variable A holds 10 and variable B holds 20, then –

Note – Addition operator (+) works for Numeric as well as Strings. e.g. "a" + 10 will give "a10".

Sr.No.	Operator & Description
1	<b>+ (Addition)</b> Adds two operands <b>Ex:</b> A + B will give 30
2	<b>- (Subtraction)</b> Subtracts the second operand from the first <b>Ex:</b> A - B will give -10
3	<b>* (Multiplication)</b> Multiply both operands <b>Ex:</b> A * B will give 200
4	<b>/ (Division)</b> Divide the numerator by the denominator <b>Ex:</b> B / A will give 2
5	<b>% (Modulus)</b> Outputs the remainder of an integer division <b>Ex:</b> B % A will give 0
6	<b>++ (Increment)</b> Increases an integer value by one <b>Ex:</b> A++ will give 11
7	<b>-- (Decrement)</b> Decreases an integer value by one <b>Ex:</b> A-- will give 9

## Example

The following code shows how to use arithmetic operators in JavaScript.

```
<html>
<body>

<script type = "text/javascript">
 <!--
 var a = 33;
 var b = 10;
 var c = "Test";
 var linebreak = "
";

 document.write("a + b = ");
 result = a + b;
 document.write(result);
 document.write(linebreak);

 document.write("a - b = ");
 result = a - b;
 document.write(result);
 document.write(linebreak);

 document.write("a / b = ");
 result = a / b;
 document.write(result);
 document.write(linebreak);
```

```
document.write("a % b = ");
 result = a % b;
 document.write(result);
 document.write(linebreak);

 document.write("a + b + c = ");
 result = a + b + c;
 document.write(result);
 document.write(linebreak);

 a = ++a;
 document.write(++a = ");
 result = ++a;
 document.write(result);
 document.write(linebreak);

 b = --b;
 document.write("--b = ");
 result = --b;
 document.write(result);
 document.write(linebreak);
 //-->
</script>

 Set the variables to different values and then try...
</body>
</html>
```

## ❑ Comparison Operators

Sr.No.	Operator & Description
1	<b>== (Equal)</b> Checks if the value of two operands are equal or not, if yes, then the condition becomes true. <b>Ex:</b> (A == B) is not true.
2	<b>!= (Not Equal)</b> Checks if the value of two operands are equal or not, if the values are not equal, then the condition becomes true. <b>Ex:</b> (A != B) is true.
3	<b>&gt; (Greater than)</b> Checks if the value of the left operand is greater than the value of the right operand, if yes, then the condition becomes true. <b>Ex:</b> (A > B) is not true.
4	<b>&lt; (Less than)</b> Checks if the value of the left operand is less than the value of the right operand, if yes, then the condition becomes true. <b>Ex:</b> (A < B) is true.
5	<b>&gt;= (Greater than or Equal to)</b> Checks if the value of the left operand is greater than or equal to the value of the right operand, if yes, then the condition becomes true. <b>Ex:</b> (A >= B) is not true.
6	<b>&lt;= (Less than or Equal to)</b> Checks if the value of the left operand is less than or equal to the value of the right operand, if yes, then the condition becomes true. <b>Ex:</b> (A <= B) is true.

### Example

The following code shows how to use comparison operators in JavaScript.

```
<html>
<body>
 <script type = "text/javascript">
 <!--
 var a = 10;
 var b = 20;
 var linebreak = "
";

 document.write("(a == b) => ");
 result = (a == b);
 document.write(result);
 document.write(linebreak);

 document.write("(a < b) => ");
 result = (a < b);
 document.write(result);
 document.write(linebreak);

 document.write("(a > b) => ");
 result = (a > b);
 document.write(result);
 document.write(linebreak);

 document.write("(a != b) => ");
 result = (a != b);
 document.write(result);
 document.write(linebreak);
 </script>
 </body>
</html>
```

```
document.write("(a >= b) => ");
 result = (a >= b);
 document.write(result);

 document.write(linebreak);

 document.write("(a <= b)
=> ");
 result = (a <= b);
 document.write(result);

 document.write(linebreak);
 <!-->
 </script>
 Set the variables to different
 values and different operators
 and then try...
 </body>
</html>
```

## ❑ Logical Operators

Sr.No.	Operator & Description
1	<b>&amp;&amp; (Logical AND)</b> If both the operands are non-zero, then the condition becomes true. <b>Ex:</b> (A && B) is true.
2	<b>   (Logical OR)</b> If any of the two operands are non-zero, then the condition becomes true. <b>Ex:</b> (A    B) is true.
3	<b>! (Logical NOT)</b> Reverses the logical state of its operand. If a condition is true, then the Logical NOT operator will make it false. <b>Ex:</b> ! (A && B) is false.

```
<html>
<body>
 <script type = "text/javascript">
 <!--
 var a = true;
 var b = false;
 var linebreak = "
";

 document.write("(a && b) => ");
 result = (a && b);
 document.write(result);
 document.write(linebreak);

 document.write("(a || b) => ");
 result = (a || b);
 document.write(result);
 document.write(linebreak);

 document.write("! (a && b) => ");
 result = (!(a && b));
 document.write(result);
 document.write(linebreak);
 //-->
 </script>
 <p>Set the variables to different values and different operators and then
 try...</p>
</body>
</html>
```

❑ Bitwise Operators

Sr.No.	Operator & Description
1	<b>&amp; (Bitwise AND)</b> It performs a Boolean AND operation on each bit of its integer arguments. <b>Ex:</b> (A & B) is 2.
2	<b>  (BitWise OR)</b> It performs a Boolean OR operation on each bit of its integer arguments. <b>Ex:</b> (A   B) is 3.
3	<b>^ (Bitwise XOR)</b> It performs a Boolean exclusive OR operation on each bit of its integer arguments. Exclusive OR means that either operand one is true or operand two is true, but not both. <b>Ex:</b> (A ^ B) is 1.
4	<b>~ (Bitwise Not)</b> It is a unary operator and operates by reversing all the bits in the operand. <b>Ex:</b> (~B) is -4.
5	<b>&lt;&lt; (Left Shift)</b> It moves all the bits in its first operand to the left by the number of places specified in the second operand. New bits are filled with zeros. Shifting a value left by one position is equivalent to multiplying it by 2, shifting two positions is equivalent to multiplying by 4, and so on. <b>Ex:</b> (A << 1) is 4.
6	<b>&gt;&gt; (Right Shift)</b> Binary Right Shift Operator. The left operand's value is moved right by the number of bits specified by the right operand. <b>Ex:</b> (A >> 1) is 1.
7	<b>&gt;&gt;&gt; (Right shift with Zero)</b> This operator is just like the >> operator, except that the bits shifted in on the left are always zero. <b>Ex:</b> (A >>> 1) is 1.

## JavaScript Assignment Operators

operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

## Shift Assignment Operators

Operator	Example	Same As
<<=	x <<= y	x = x << y
>>=	x >>= y	x = x >> y
>>>=	x >>>= y	x = x >>> y

## Logical Assignment Operators

Operator	Example	Same As
&&=	x &&= y	x = x && (x = y)
=	x   = y	x = x    (x = y)
??=	x ??= y	x = x ?? (x = y)

```

<html>
<body>
 <script type = "text/javascript">
 <!--
 var a = 2; // Bit presentation 10
 var b = 3; // Bit presentation 11
 var linebreak = "
";

 document.write("(a & b) => ");
 result = (a & b);
 document.write(result);
 document.write(linebreak);

 document.write("(a | b) => ");
 result = (a | b);
 document.write(result);
 document.write(linebreak);

 document.write("(a ^ b) => ");
 result = (a ^ b);
 document.write(result);
 document.write(linebreak);
 </script>
 </body>
</html>

```

```

document.write("(~b) => ");
 result = (~b);
 document.write(result);
 document.write(linebreak);

 document.write("(a << b) => ");
 result = (a << b);
 document.write(result);
 document.write(linebreak);

 document.write("(a >> b) => ");
 result = (a >> b);
 document.write(result);
 document.write(linebreak);
 //-->
</script>
<p>Set the variables to different values and different operators
and then try...</p>
</body>
</html>

```

## ❑ Assignment Operators

Sr.No.	Operator & Description
1	<b>= (Simple Assignment )</b> Assigns values from the right side operand to the left side operand <b>Ex:</b> $C = A + B$ will assign the value of $A + B$ into $C$
2	<b>+= (Add and Assignment)</b> It adds the right operand to the left operand and assigns the result to the left operand. <b>Ex:</b> $C += A$ is equivalent to $C = C + A$
3	<b>-= (Subtract and Assignment)</b> It subtracts the right operand from the left operand and assigns the result to the left operand. <b>Ex:</b> $C -= A$ is equivalent to $C = C - A$
4	<b>*= (Multiply and Assignment)</b> It multiplies the right operand with the left operand and assigns the result to the left operand. <b>Ex:</b> $C *= A$ is equivalent to $C = C * A$
5	<b>/= (Divide and Assignment)</b> It divides the left operand with the right operand and assigns the result to the left operand. <b>Ex:</b> $C /= A$ is equivalent to $C = C / A$
6	<b>%= (Modules and Assignment)</b> It takes modulus using two operands and assigns the result to the left operand. <b>Ex:</b> $C \% = A$ is equivalent to $C = C \% A$

**Note** – Same logic applies to Bitwise operators so they will become like  $<<=$ ,  $>>=$ ,  $>>=$ ,  $\&=$ ,  $|=$  and  $\wedge=$ .

## Example

Try the following code to implement assignment operator in JavaScript.

```
<html>
<body>
 <script type = "text/javascript">
 <!--
 var a = 33;
 var b = 10;
 var linebreak = "
";

 document.write("Value of a => (a = b) => ");
 result = (a = b);
 document.write(result);
 document.write(linebreak);

 document.write("Value of a => (a += b) => ");
 result = (a += b);
 document.write(result);
 document.write(linebreak);

 document. Write("Value of a => (a -= b) => ");
 result = (a -= b);
 document.write(result);
 document.write(linebreak);
```

```
 document.write("Value of a => (a *= b) => ");
 result = (a *= b);
 document.write(result);
 document.write(linebreak);

 document.write("Value of a => (a /= b) => ");
 result = (a /= b);
 document.write(result);
 document.write(linebreak);

 document.write("Value of a => (a %= b) => ");
 result = (a %= b);
 document.write(result);
 document.write(linebreak);
 //-->
 </script>
 <p>Set the variables to different values and different operators
 and then try...</p>
</body>
</html>
```

## ❑ Conditional Operator (? :)

The conditional operator first evaluates an expression for a true or false value and then executes one of the two given statements depending upon the result of the evaluation.

? : (Conditional )

If Condition is true? Then value X : Otherwise value Y

```
<html>
 <body>
 <script type = "text/javascript">
 <!--
 var a = 10;
 var b = 20;
 var linebreak = "
";

 document.write ("((a > b) ? 100 : 200) => ");
 result = (a > b) ? 100 : 200;
 document.write(result);
 document.write(linebreak);

 document.write ("((a < b) ? 100 : 200) => ");
 result = (a < b) ? 100 : 200;
 document.write(result);
 document.write(linebreak);
 //-->
 </script>
 <p>Set the variables to different values and different
operators and then try...</p>
 </body>
</html>
```

## typeof Operator

The **typeof** operator is a unary operator that is placed before its single operand, which can be of any type. Its value is a string indicating the data type of the operand.

The *typeof* operator evaluates to "number", "string", or "boolean" if its operand is a number, string, or boolean value and returns true or false based on the evaluation. Here is a list of the return values for the **typeof** Operator.

Type	String Returned by typeof
Number	"number"
String	"string"
Boolean	"boolean"
Object	"object"
Function	"function"
Undefined	"undefined"
Null	"object"

```
<html>
```

```
<body>
```

```
<script type = "text/javascript">
```

```
<!--
```

```
var a = 10;
```

```
var b = "String";
```

```
var linebreak = "
";
```

```
result = (typeof b == "string" ? "B is String" : "B is
Numeric");
```

```
document.write("Result => ");
```

```
document.write(result);
```

```
document.write(linebreak);
```

```
result = (typeof a == "string" ? "A is String" : "A is
Numeric");
```

```
document.write("Result => ");
```

```
document.write(result);
```

```
document.write(linebreak);
```

```
//-->
```

```
</script>
```

```
<p>Set the variables to different values and different operators
and then try...</p>
```

```
</body>
```

```
</html>
```

## ❑ JavaScript - if...else Statement

JavaScript supports conditional statements which are used to perform different actions based on different conditions. Here we will explain the if..else statement.

JavaScript supports the following forms of if..else statement –

if statement

if...else statement

if...else if... statement.

### Syntax

The syntax for a basic if statement is as follows –

```
if (expression) {
 Statement(s) to be executed if expression is true
}
```

```
<html>
 <body>
 <script type = "text/javascript">
 <!--
 var age = 20;

 if(age > 18) {
 document.write("Qualifies for
driving");
 }
 //-->
 </script>
 <p>Set the variable to different value and then
try...</p>
 </body>
</html>
```

```

<html>
 <body>
 <script type = "text/javascript">
 <!--
 var age = 20;

 if(age > 18) {
 document.write("Qualifies for
driving");
 }
 //-->
 </script>
 <p>Set the variable to different value and
then try...</p>
 </body>
</html>

```

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript if</h2>

<p>Display "Good day!" if the hour is less than
18:00:</p>

<p id="demo">Good Evening!</p>

<script>
if (new Date().getHours() < 18) {
 document.getElementById("demo").innerHTML =
"Good day!";
}
</script>

</body>
</html>

```









## ❑ if...else statement

### Syntax

```
if (expression) {
 Statement(s) to be executed if expression is true
} else {
 Statement(s) to be executed if expression is false
}
```

## ❑ if...else if... statement

### Syntax

The syntax of an if-else-if statement is as follows –

```
if (expression 1) {
 Statement(s) to be executed if expression 1 is true
} else if (expression 2) {
 Statement(s) to be executed if expression 2 is true
} else if (expression 3) {
 Statement(s) to be executed if expression 3 is true
} else {
 Statement(s) to be executed if no expression is true
}
```

html>

<body>

<script type = "text/javascript">

<!--

var book = "maths";

if( book == "history" ) {

document.write("<b>History Book</b>");

} else if( book == "maths" ) {

document.write("<b>Maths Book</b>");

} else if( book == "economics" ) {

document.write("<b>Economics Book</b>");

} else {

document.write("<b>Unknown Book</b>");

}

//-->

</script>

<p>Set the variable to different value and then

try...</p>

</body>

<html>

```

<html>
<body>
 <script type = "text/javascript">
 <!--
 var grade = 'A';
 document.write("Entering switch block
");
 switch (grade) {
 case 'A': document.write("Good job
");
 break;

 case 'B': document.write("Pretty good
");
 break;

 case 'C': document.write("Passed
");
 break;

 case 'D': document.write("Not so good
");
 break;

 case 'F': document.write("Failed
");
 break;

 default: document.write("Unknown grade
")
 }
 document.write("Exiting switch block");
 //-->
 </script>
 <p>Set the variable to different value and then try...</p>
</body>
</html>

```

```

<html>
<body>
 <script type = "text/javascript">
 <!--
 var grade = 'A';
 document.write("Entering switch block
");
 switch (grade) {
 case 'A': document.write("Good job
");
 case 'B': document.write("Pretty good
");
 case 'C': document.write("Passed
");
 case 'D': document.write("Not so good
");
 case 'F': document.write("Failed
");
 default: document.write("Unknown grade
")
 }
 document.write("Exiting switch block");
 //-->
 </script>
 <p>Set the variable to different value and then
try...</p>
</body>
</html>

```

## JavaScript Switch

The JavaScript switch statement is used to execute one code from multiple expressions. It is just like else if statement that we have learned in previous page. But it is convenient than if..else..if because it can be used with numbers, characters etc.

```
switch(expression){
 case value1:
 code to be executed;
 break;
 case value2:
 code to be executed;
 break;

 default:
 code to be executed if above values are not matched;
}
```

```
<!DOCTYPE html>
<html>
<body>
<script>
var grade='B';
var result;
switch(grade){
 case 'A':
 result="A Grade";
 break;
 case 'B':
 result="B Grade";
 break;
 case 'C':
 result="C Grade";
 break;
 default:
 result="No Grade";
 }
 document.write(result);
</script>
</body>
</html>
```

## ❑ JavaScript Loops

The JavaScript loops are used to iterate the piece of code using for, while, do while or for-in loops. It makes the code compact. It is mostly used in array.

There are four types of loops in JavaScript.

- for loop
- while loop
- do-while loop
- for-in loop

```
<!DOCTYPE html>
<html>
<body>
<script>
for (i=1; i<=5; i++)
{
document.write(i + "
")
}
</script>
</body>
</html>
```

### 1) JavaScript For loop

The JavaScript for loop iterates the elements for the fixed number of times. It should be used if number of iteration is known. The syntax of for loop is given below.

```
for (initialization; condition; increment)
{
 code to be executed
}
```

### 2) JavaScript while loop

The JavaScript while loop iterates the elements for the infinite number of times. It should be used if number of iteration is not known. The syntax of while loop is given below.

```
while (condition)
{
 code to be executed
}
```

```
<!DOCTYPE html>
<html>
<body>
<script>
var i=11;
while (i<=15)
{
document.write(i + "
");
i++;
}
</script>
</body>
</html>
```

### 3) JavaScript do while loop

The JavaScript do while loop iterates the elements for the infinite number of times like while loop. But, code is executed at least once whether condition is true or false. The syntax of do while loop is given below.

```
do{
 code to be executed
}while (condition);
```

```
<!DOCTYPE html>
<html>
<body>
<script>
var i=21;
do{
document.write(i + "
");
i++;
}while (i<=25);
</script>
</body>
</html>
```

## 4.4. Functions; Popup Boxes: Alert, Confirm, Prompt

➤ JavaScript Message Boxes: alert(), confirm(), prompt()  
JavaScript provides built-in global functions to display popup message boxes for different purposes.

1. alert(message): Display a popup box with the specified message with the OK button.
2. confirm(message): Display a popup box with the specified message with OK and Cancel buttons.
3. prompt(message, defaultValue): Display a popup box to take the user's input with the OK and Cancel buttons.

Note:

In JavaScript, global functions can be accessed using the window object like window.alert(), window.confirm(), window.prompt().

### 1. alert()

➤ The alert() function displays a message to the user to display some information to users. This alert box will have the OK button to close the alert box.

### Example: alert()

```
<!DOCTYPE html>
<html>
<body>
 <h1>Demo: alert()</h1>
 <button onclick="displayInfo()">Display
Messages</button>

 <script>
 function displayInfo(){
 alert("This is an alert message
box."); // display string message

 alert("This is a numer: ' + 100);
// display result of a concatenation

 alert(100); // display number

 alert(Date()); // display current date
 }

 </script>
</body>
</html>
```

## 2. confirm()

- Use the confirm() function to take the user's confirmation before starting some task. For example, you want to take the user's confirmation before saving, updating or deleting data.

Syntax:

```
bool window.confirm([message]);
```

- The confirm() function displays a popup message to the user with two buttons, OK and Cancel. The confirm() function returns true if a user has clicked on the OK button or returns false if clicked on the Cancel button. You can use the return value to process further.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
 <h1>Demo: confirm() </h1>
```

```
 <button onclick="save()">Save Data</button>
```

```
 <p id="msg"></p>
```

```
<script>
```

```
 function save(){
 var userPreference;
```

```
 if (confirm("Do you want to save
changes?") == true) {
 userPreference = "Data saved successfully!";
 }
```

```
 else {
 userPreference = "Save Canceled!";
 }
```

```
 document.getElementById("msg").innerHTML = userPreference;
}
```

```
</script>
```

```
</body>
```

```
</html>
```

## prompt()

- Use the prompt() function to take the user's input to do further actions. For example, use the prompt() function in the scenario where you want to calculate EMI based on the user's preferred loan tenure.

Syntax:

```
string prompt([message], [defaultValue]);
```

- The prompt() function takes two parameters. The first parameter is the message to be displayed, and the second parameter is the default value in an input box.

```
<!DOCTYPE html>
<html>
<body>
 <h1>Demo: prompt()</h1>
 <button onclick="myinput()">Click to enter your
name</button>
 <p id="msg"></p>
```

```
<script>
 function myinput(){
 var name = prompt("Enter Your
Name:");

 if (name == null || name == "") {

 document.getElementById("msg").innerHTML = "You
did not enter anything. Please enter your name again";
 }
 else
 {

 document.getElementById("msg").innerHTML = "You
entered: " + name;
 }
 }

</script>
</body>
</html>
```

## 4.5. Objects and properties; Constructors

- JavaScript is an Object Oriented Programming (OOP) language. A programming language can be called object-oriented if it provides four basic capabilities to developers –
  - ❑ Encapsulation – the capability to store related information, whether data or methods, together in an object.
  - ❑ Aggregation – the capability to store one object inside another object.
  - ❑ Inheritance – the capability of a class to rely upon another class (or number of classes) for some of its properties and methods.
  - ❑ Polymorphism – the capability to write one function or method that works in a variety of different ways.
- JavaScript is template based not class based. Here, we don't create class to get the object. But, we direct create objects.

## ❑ Creating Objects in JavaScript

There are 3 ways to create objects.

1. By object literal
2. By creating instance of Object directly (using new keyword)
3. By using an object constructor (using new keyword)

### 1) JavaScript Object by object literal

- The syntax of creating object using object literal is given below:  
object={property1:value1,property2:value2.....propertyN:valueN}
- As you can see, property and value is separated by : (colon).
- Let's see the simple example of creating object in JavaScript.

```
<html>
<body>
<script>
emp={id:102,name:"Shyam Kumar",salary:40000}
document.write(emp.id+" "+emp.name+" "+emp.salary);
</script>
</body>
</html>
```

## 2) By creating instance of Object

The syntax of creating object directly is given below:

```
var objectname=new Object();
```

Here, new keyword is used to create object.

Let's see the example of creating object directly.

```
<html>
<body>
<script>
var emp=new Object();
emp.id=101;
emp.name="Ravi Malik";
emp.salary=50000;
document.write(emp.id+" "+emp.name+" "+emp.salary);
</script>
</body>
</html>
```

## 3) By using an Object constructor

- Here, you need to create function with arguments. Each argument value can be assigned in the current object by using this keyword.
- The **this keyword** refers to the current object.
- The example of creating object by object constructor is given below.

```
<html>
<body>
<script>
function emp(id,name,salary){
this.id=id;
this.name=name;
this.salary=salary;
}
e=new emp(103,"Vimal Jaiswal",30000);

document.write(e.id+" "+e.name+" "+e.salary);
</script>
</body>
</html>
```

## ❑ Defining method in JavaScript Object

We can define method in JavaScript object. But before defining method, we need to add property in the function with same name as method.

The example of defining method in object is given below.

```
<html>
<body>
<script>
function emp(id,name,salary){
this.id=id;
this.name=name;
this.salary=salary;

this.changeSalary=changeSalary;
function changeSalary(otherSalary){
this.salary=otherSalary;
}
}
e=new emp(103,"Sonoo Jaiswal",30000);
document.write(e.id+" "+e.name+" "+e.salary);
e.changeSalary(45000);
document.write("
" +e.id+" "+e.name+" "+e.salary);
</script>
</body>
</html>
```

S.No	Methods	Description
1	<a href="#">Object.assign()</a>	This method is used to copy enumerable and own properties from a source object to a target object
2	<a href="#">Object.create()</a>	This method is used to create a new object with the specified prototype object and properties.
3	<a href="#">Object.defineProperty()</a>	This method is used to describe some behavioral attributes of the property.
4	<a href="#">Object.defineProperties()</a>	This method is used to create or configure multiple object properties.
5	<a href="#">Object.entries()</a>	This method returns an array with arrays of the key, value pairs.
6	<a href="#">Object.freeze()</a>	This method prevents existing properties from being removed.
7	<a href="#">Object.getOwnPropertyDescriptor()</a>	This method returns a property descriptor for the specified property of the specified object.
8	<a href="#">Object.getOwnPropertyDescriptors()</a>	This method returns all own property de

## 4.6. Arrays

**JavaScript array** is an object that represents a collection of similar type of elements.

There are 3 ways to construct array in JavaScript

- 1.By array literal
- 2.By creating instance of Array directly (using new keyword)
- 3.By using an Array constructor (using new keyword)

### 2) JavaScript Array directly (new keyword)

The syntax of creating array directly is given below:

```
var arrayname=new Array();
```

Here, **new keyword** is used to create instance of array.

### JavaScript array literal

The syntax of creating array using array literal is given below:

```
var arrayname=[value1,value2.....valueN];
```

As you can see, values are contained inside [ ] and separated by , (comma).

### 3) JavaScript array constructor (new keyword)

Here, you need to create instance of array by passing arguments in constructor so that we don't have to provide value explicitly.

## 4.6. Arrays example

```
<html>
<body>
<script>
var emp=["Sonoo","Vimal","Ratan"];
for (i=0;i<emp.length;i++)
{
document.write(emp[i] + "
");
}
</script>
</body>
</html>
```

```
<html>
<body>
<script>
var emp=new Array("Jai","Vijay","Smith");
for (i=0;i<emp.length;i++){
document.write(emp[i] + "
");
}
</script>
</body>
</html>
```

```
<html>
<body>
<script>
var i;
var emp = new Array();
emp[0] = "Arun";
emp[1] = "Varun";
emp[2] = "John";

for (i=0;i<emp.length;i++){
document.write(emp[i] + "
");
}
</script>
</body>
</html>
```

## 4.7. Built-in Objects: Window, String, Number, Boolean, Date, Math, RegExp, Form, DOM

### JavaScript String

The **JavaScript string** is an object that represents a sequence of characters.

There are 2 ways to create string in JavaScript

- 1.By string literal
- 2.By string object (using new keyword)

#### 1) By string literal

The string literal is created using double quotes. The syntax of creating string using string literal is given below:

```
var stringname="string value";
```

```
<!DOCTYPE html>
<html>
<body>
<script>
var str="This is string literal";
document.write(str);
</script>
</body>
</html>
```

#### 2) By string object (using new keyword)

The syntax of creating string object using new keyword is given below:

```
1.var stringname=new String("string literal");
```

Here, **new keyword** is used to create instance of string.

```
<!DOCTYPE html>
<html>
<body>
<script>
var stringname=new String("hello javascript string");
document.write(stringname);
</script>
</body>
</html>
```

## JavaScript Date Object

The JavaScript date object can be used to get year, month and day. You can display a timer on the webpage by the help of JavaScript date object.

You can use different Date constructors to create date object. It provides methods to get and set day, month, year, hour, minute and seconds.

### Constructor

You can use 4 variant of Date constructor to create date object.

Date()

Date(milliseconds)

Date(dateString)

Date(year, month, day, hours, minutes, seconds, milliseconds)

## JavaScript Date Methods

Let's see the list of JavaScript date methods with their description.

Methods	Description
<a href="#">getDate()</a>	It returns the integer value between 1 and 31 that represents the day for the specified date on the basis of local time.
<a href="#">getDay()</a>	It returns the integer value between 0 and 6 that represents the day of the week on the basis of local time.
<a href="#">getFullYear()</a>	It returns the integer value that represents the year on the basis of local time.
<a href="#">getHours()</a>	It returns the integer value between 0 and 23 that represents the hours on the basis of local time.
<a href="#">getMilliseconds()</a>	It returns the integer value between 0 and 999 that represents the milliseconds on the basis of local time.
<a href="#">getMinutes()</a>	It returns the integer value between 0 and 59 that represents the minutes on the basis of local time.
<a href="#">getMonth()</a>	It returns the integer value between 0 and 11 that represents the month on the basis of local time.
<a href="#">getSeconds()</a>	It returns the integer value between 0 and 60 that represents the seconds on the basis of local time.
<a href="#">getUTCDate()</a>	It returns the integer value between 1 and 31 that represents the day for the specified date on the basis of universal time.
<a href="#">getUTCDay()</a>	It returns the integer value between 0 and 6 that represents the day of the week on the basis of universal time.

```
<html>
<body>
Current Date and Time:
<script>
var today=new Date();
document.getElementById('txt').innerHTML=today;
</script>
</body>
</html>
```

```
<script>
var date=new Date();
var day=date.getDate();
var month=date.getMonth()+1;
var year=date.getFullYear();
document.write("
Date is:
"+day+"/"+month+"/"+year);
</script>
```

```
<html>
<body>
Current Time:
<script>
var today=new Date();
var h=today.getHours();
var m=today.getMinutes();
var s=today.getSeconds();
document.getElementById('txt').innerHTML=h+"
:"+m+": "+s;
</script>
</body>
</html>
```

```

<html>
<body>
Current Time:
<script>
window.onload=function(){getTime();}
function getTime(){
var today=new Date();
var h=today.getHours();
var m=today.getMinutes();
var s=today.getSeconds();
// add a zero in front of numbers<10
m=checkTime(m);
s=checkTime(s);
document.getElementById('txt').innerHTML=h+":"+m+":"+s;
setTimeout(function(){getTime()},1000);
}
//setInterval("getTime()",1000);//another way
function checkTime(i){
if (i<10){
i="0" + i;
}
return i;
}
</script>
</body>
</html>

```

## JavaScript Number Object

The JavaScript number object enables you to represent a numeric value. It may be integer or floating-point. JavaScript number object follows IEEE standard to represent the floating-point numbers.

By the help of Number() constructor, you can create number object in JavaScript. For example:

```
var n=new Number(value);
```

If value can't be converted to number, it returns NaN(Not a Number) that can be checked by isNaN() method.

```

<!DOCTYPE html>
<html>
<body>
<script>
var x=102;//integer value
var y=102.7;//floating point value
var z=13e4;//exponent value, output: 130000
var n=new Number(16);//integer value by number object
document.write(x+" "+y+" "+z+" "+n);
</script>
</body>
</html>

```

## JavaScript Number Constants

Let's see the list of JavaScript number constants with description.

Constant	Description
MIN_VALUE	returns the largest minimum value.
MAX_VALUE	returns the largest maximum value.
POSITIVE_INFINITY	returns positive infinity, overflow value.
NEGATIVE_INFINITY	returns negative infinity, overflow value.
NaN	represents "Not a Number" value.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Numbers</h1>
```

```
<h2>The MAX_VALUE Property</h2>
```

```
<p>Number.MAX_VALUE returns the largest possible number in
JavaScript:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML =
Number.MAX_VALUE;
```

```
</script>
```

```
</body>
```

```
</html>
```

## JavaScript Boolean

**JavaScript Boolean** is an object that represents value in two states: *true* or *false*. You can create the JavaScript Boolean object by Boolean() constructor as given below.

1.Boolean **b**=new Boolean(value);

The default value of JavaScript Boolean object is *false*.

## JavaScript Boolean Example

```
<script>
document.write(10<20);//true
document.write(10<5);//false
</script>
```

```
<!DOCTYPE html>
<html>
```

```
<body>
```

```
<h2>JavaScript Booleans</h2>
```

```
<p>Display the value of Boolean(10 > 9):</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = Boolean(10 > 9);
```

```
</script>
```

```
</body>
```

```
</html>
```

## JavaScript Boolean Properties

Property	Description
constructor	returns the reference of Boolean function that created Boolean object.
prototype	enables you to add properties and methods in Boolean prototype.

## JavaScript Boolean Methods

Method	Description
toSource()	returns the source of Boolean object as a string.
toString()	converts Boolean into String.
valueOf()	converts other type into Boolean.

## Everything With a "Value" is True

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Booleans</h2>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
"100 is " + Boolean(100) + "
" +
"3.14 is " + Boolean(3.14) + "
" +
"-15 is " + Boolean(-15) + "
" +
"Any (not empty) string is " + Boolean("Hello") +
"
" +
"Even the string 'false' is " + Boolean('false') + "
"
+
"Any expression (except zero) is " + Boolean(1 + 7 +
3.14);
</script>

</body>
</html>
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Booleans</h2>
<p>Booleans and Boolean objects cannot be safely compared:</p>

<p id="demo"></p>

<script>
// x is a boolean
let x = false;

// y is an object
let y = new Boolean(false);

document.getElementById("demo").innerHTML = typeof x + "
" +
typeof y;
</script>

</body>
</html>
```

## 4.8. User Defined Objects; Event Handling and Form Validation

### JavaScript Events

- The change in the state of an object is known as an **Event**. In html, there are various events which represents that some activity is performed by the user or by the browser. When **javascript** code is included in [HTML](#)
- js react over these events and allow the execution. This process of reacting over the events is called **Event Handling**. Thus, js handles the HTML events via **Event Handlers**.
- **For example**, when a user clicks over the browser, add js code, which will execute the task to be performed on the event.

Some of the HTML events and their event handlers are:

#### Mouse events:

Event Performed	Event Handler	Description
click	onclick	When mouse click on an element
mouseover	onmouseover	When the cursor of the mouse comes over the element
mouseout	onmouseout	When the cursor of the mouse leaves an element
mousedown	onmousedown	When the mouse button is pressed over the element
mouseup	onmouseup	When the mouse button is released over the element
mousemove	onmousemove	When the mouse movement takes place.

### Keyboard events:

Event Performed	Event Handler	Description
Keydown & Keyup	onkeydown & onkeyup	When the user press and then release the key

### Form events:

Event Performed	Event Handler	Description
focus	onfocus	When the user focuses on an element
submit	onsubmit	When the user submits the form
blur	onblur	When the focus is away from a form element
change	onchange	When the user modifies or changes the value of a form element

### Window/Document events

Event Performed	Event Handler	Description
load	onload	When the browser finishes the loading of the page
unload	onunload	When the visitor leaves the current webpage, the browser unloads it
resize	onresize	When the visitor resizes the window of the browser

## Click Event example

```
<html>
<head> Javascript Events </head>
<body>
<script language="Javascript" type="text/Javascript">
 <!--
 function clickevent()
 {
 document.write("This is JavaTpoint");
 }
 //-->
</script>
<form>
<input type="button" onclick="clickevent()" value="Who's
this?"/>
</form>
</body>
</html>
```

## MouseOver Event

```
<html>
<head>
<h1> Javascript Events </h1>
</head>
<body>
<script language="Javascript" type="text/Javascript">
 <!--
 function mouseoverevent()
 {
 alert("This is JavaTpoint");
 }
 //-->
</script>
<p onmouseover="mouseoverevent()"> Keep cursor over
me</p>
</body>
</html>
```

## Keydown Event

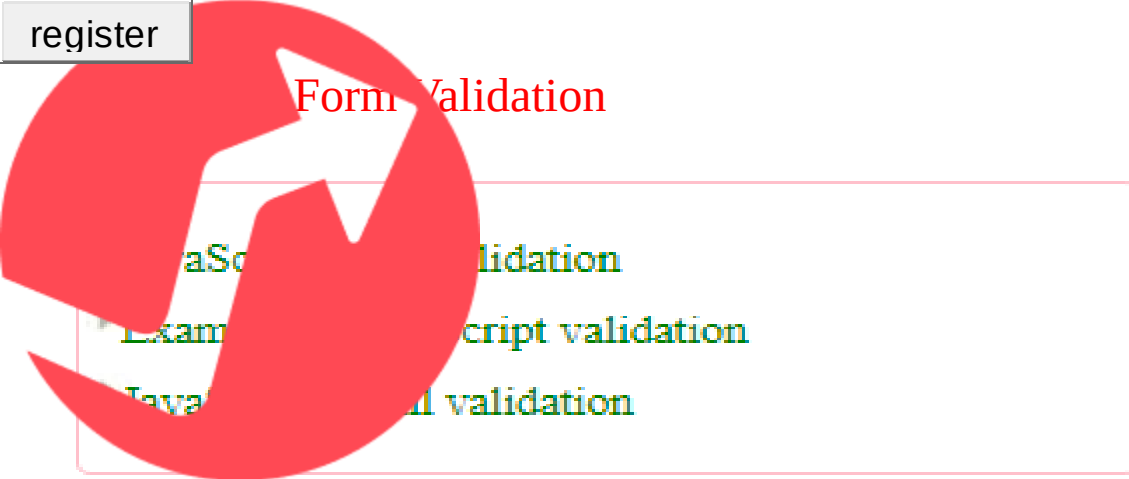
```
<html>
<head> Javascript Events</head>
<body>
<h2> Enter something here</h2>
<input type="text" id="input1"
onkeydown="keydownevent()"/>
<script>
<!--
 function keydownevent()
 {

 document.getElementById("input1");
 alert("Pressed a key");

 }
//-->
</script>
</body>
</html>
```

## JavaScript addEventListener()

```
<!DOCTYPE html>
<html>
<body>
<p> Example of the addEventListener() method. </p>
<p> Click the following button to see the effect. </p>
<button id = "btn"> Click me </button>
<p id = "para"></p>
<script>
document.getElementById("btn").addEventListener("click", fun);
function fun() {
document.getElementById("para").innerHTML = "Hello World" +
"
" + "Welcome to the Web Technology class.com";
}
</script>
</body>
</html>
```



## JavaScript Form Validation Example

In this example, we are going to validate the name and password. The name can't be empty and password can't be less than 6 characters long.

Here, we are validating the form on form submit. The user will not be forwarded to the next page until given values are correct.

It is important to validate the form submitted by the user because it can have inappropriate values. So, validation is must to authenticate user.

JavaScript provides facility to validate the form on the client-side so data processing will be faster than server-side validation. Most of the web developers prefer JavaScript form validation.

Through JavaScript, we can validate name, password, email, date, mobile numbers and more fields.

## JavaScript Form Validation Example

```
<html>
<body>
<script>
function validateform(){
var name=document.myform.name.value;
var password=document.myform.password.value;

if (name==null || name==""){
 alert("Name can't be blank");
 return false;
}else if(password.length<6){
 alert("Password must be at least 6 characters long.");
 return false;
}
}
</script>
<body>
<form name="myform" method="post"
action="http://www.javatpoint.com/javascriptpages/valid.jsp"
onsubmit="return validateform()" >
Name: <input type="text" name="name">

Password: <input type="password" name="password">

<input type="submit" value="register">
</form>
</body>
</html>
```

## JavaScript Retype Password Validation

```
<!DOCTYPE html>
<html>
<head>
<script type="text/javascript">
function matchpass(){
var firstpassword=document.f1.password.value;
var secondpassword=document.f1.password2.value;

if(firstpassword==secondpassword){
 return true;
}
else{
 alert("password must be same!");
 return false;
}
}
</script>
</head>
<body>

<form name="f1"
action="http://www.javatpoint.com/javascriptpages/valid.jsp"
onsubmit="return matchpass()">
Password:<input type="password" name="password" />

Re-enter Password:<input type="password" name="password2"/>

<input type="submit">
</form>

</body>
</html>
```

## JavaScript Number Validation

```
<!DOCTYPE html>
<html>
<head>
<script>
function validate(){
var num=document.myform.num.value;
if (isNaN(num)){
 document.getElementById("numloc").innerHTML="Enter Numeric value
only";
 return false;
}else{
 return true;
}
}
</script>
</head>

<body>
<form name="myform"
action="http://www.javatpoint.com/javascriptpages/valid.jsp"
onsubmit="return validate()" >
Number: <input type="text" name="num">

<input type="submit" value="submit">
</form>

</body>
</html>
```

## JavaScript validation with image

```
<html>
<body>
<script type="text/javascript">
function validate(){
var name=document.f1.name.value;
var passwordlength=document.f1.password.value.length;
var status=false;
if(name==""){
document.getElementById("namelocation").innerHTML=
"
Please enter your name";
status=false;
}else{
document.getElementById("namelocation").innerHTML=" ";
status=true;
}

if(passwordlength<6){
document.getElementById("passwordlocation").innerHTML=
"
Password must be greater than 6";
status=false;
}else{
document.getElementById("passwordlocation").innerHTML=" ";
}
return status;
}
```

## JavaScript email validation

We can validate the email by the help of JavaScript.

There are many criteria that need to be follow to validate the email id such as:

- email id must contain the @ and . character
- There must be at least one character before and after the @.
- There must be at least two characters after . (dot).

```
<html>
<body>
<script>
function validateemail()
{
var x=document.myform.email.value;
var atposition=x.indexOf("@");
var dotposition=x.lastIndexOf(".");
if (atposition<1 || dotposition<atposition+2 || dotposition+2>=x.length){
 alert("Please enter a valid e-mail address \n atpostion:"+atposition+"\n
dotposition:"+dotposition);
 return false;
}
}
</script>
<body>
<form name="myform" method="post"
action="http://www.javatpoint.com/javascriptpages/valid.jsp"
onsubmit="return validateemail();">
Email: <input type="text" name="email">

<input type="submit" value="register">
</form>
</body>
</html>
```

## 4.9. Error Handling (Try/Catch),

### JavaScript try...catch

- A try...catch is a commonly used statement in various programming languages. Basically, it is used to handle the error-prone part of the code. It initially tests the code for all possible errors it may contain, then it implements actions to tackle those errors (if occur). A good programming approach is to keep the complex code within the try...catch statements.
- **try{} statement:** Here, the code which needs possible error testing is kept within the try block. In case any error occur, it passes to the catch{} block for taking suitable actions and handle the error. Otherwise, it executes the code written within.
- **catch{} statement:** This block handles the error of the code by executing the set of statements written within the block. This block contains either the user-defined exception handler or the built-in handler. This block executes only when any error-prone code needs to be handled in the try block. Otherwise, the catch block is skipped.

Note: catch {} statement executes only after the execution of the try {} statement. Also, one try block can contain one or more catch blocks.

### Syntax:

```
try{
expression; } //code to be written.
catch(error){
expression; } // code for handling the error.
```

```
<html>
<head> Exception Handling</br></head>
<body>
<script>
try{
var a= ["34","32","5","31","24","44","67"]; //a is an array
document.write(a); // displays elements of a
document.write(b); //b is undefined but still trying to fetch its value.
Thus catch block will be invoked
}catch(e){
alert("There is error which shows "+e.message); //Handling error
}
</script>
</body>
</html>
```

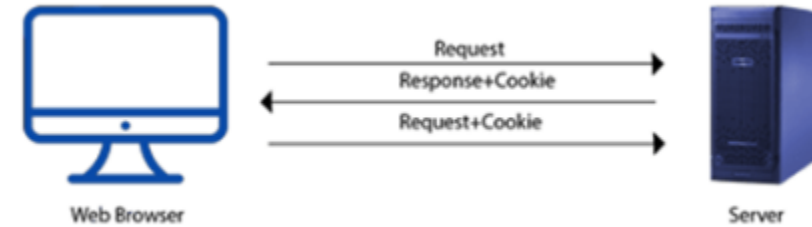
## 4.10.Handling Cookies

### JavaScript Cookies

- A cookie is an amount of information that persists between a server-side and a client-side. A web browser stores this information at the time of browsing.
- A cookie contains the information as a string generally in the form of a name-value pair separated by semi-colons. It maintains the state of a user and remembers the user's information among all the web pages.

#### ❑ How Cookies Works?

- When a user sends a request to the server, then each of that request is treated as a new request sent by the different user.
- So, to recognize the old user, we need to add the cookie with the response from the server.
- browser at the client-side.
- Now, whenever a user sends a request to the server, the cookie is added with that request automatically. Due to the cookie, the server recognizes the users.



### How to create a Cookie in JavaScript?

- In JavaScript, we can create, read, update and delete a cookie by using **document.cookie** property.
- The following syntax is used to create a cookie:

```
document.cookie="name=value";
```

## JavaScript Cookie Example

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<input type="button" value="setCookie" onclick="setCookie()">
<input type="button" value="getCookie" onclick="getCookie()">
 <script>
 function setCookie()
 {
 document.cookie="username=Duke Martin";
 }
 function getCookie()
 {
 if(document.cookie.length!=0)
 {
 alert(document.cookie);
 }
 else
 {
 alert("Cookie not available");
 }
 }
 </script>
</body>
</html>
```

## Example 2

Here, we display the cookie's name-value pair separately.

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<input type="button" value="setCookie" onclick="setCookie()">
<input type="button" value="getCookie" onclick="getCookie()">
 <script>
 function setCookie()
 {
 document.cookie="username=Duke Martin";
 }
 function getCookie()
 {
 if(document.cookie.length!=0)
 {
 var array=document.cookie.split("=");
 alert("Name="+array[0]+" "+"Value="+array[1]);
 }
 else
 {
 alert("Cookie not available");
 }
 }
 </script>
</body>
</html>
```

### Example 3

In this example, we provide choices of color and pass the selected color value to the cookie. Now, cookie stores the last choice of a user in a browser. So, on reloading the web page, the user's last choice will be shown on the screen.

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
 <select id="color" onchange="display()">
 <option value="Select Color">Select Color</option>
 <option value="yellow">Yellow</option>
 <option value="green">Green</option>
 <option value="red">Red</option>
 </select>
 <script type="text/javascript">
 function display()
 {
 var value = document.getElementById("color").value;
 if (value != "Select Color")
 {
 document.bgColor = value;
 document.cookie = "color=" + value;
 }
 }
 </script>
</body>
</html>
```

```

 }
 window.onload = function ()
 {
 if (document.cookie.length != 0)
 {
 var array = document.cookie.split("=")
 ;
 document.getElementById("color").value = array[1];
 document.bgColor = array[1];
 }
 }
 </script>
</body>
</html>
```

## Cookie Attributes

JavaScript provides some optional attributes that enhance the functionality of cookies. Here, is the list of some attributes with their description.

Attributes	Description
expires	It maintains the state of a cookie up to the specified date and time.
max-age	It maintains the state of a cookie up to the specified time. Here, time is given in seconds.
path	It expands the scope of the cookie to all the pages of a website.
domain	It is used to specify the domain for which the cookie is valid.

### Cookie expires attribute

The cookie expires attribute provides one of the ways to create a persistent cookie. Here, a date and time are declared that represents the active period of a cookie. Once the declared time is passed, a cookie is deleted automatically.

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<input type="button" value="setCookie" onclick="setCookie()">
<input type="button" value="getCookie" onclick="getCookie()">
 <script>
 function setCookie()
 {
 document.cookie="username=Duke Martin;expires=Sun, 20 Aug 2030 12:00:
00 UTC";
 }
 function getCookie()
 {
 if(document.cookie.length!=0)
 {
 var array=document.cookie.split("=");
 alert("Name="+array[0]+" "+"Value="+array[1]);
 }
 else
 {
 alert("Cookie not available");
 }
 }
 </script>
</body>
</html>
```

## Cookie max-age attribute

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<input type="button" value="setCookie" onclick="setCookie()">
<input type="button" value="getCookie" onclick="getCookie()">
 <script>
 function setCookie()
 {
 document.cookie="username=Duke Martin;max-
age=" + (60 * 60 * 24 * 365) + ";";
 }
 function getCookie()
 {
 if(document.cookie.length!=0)
 {
 var array=document.cookie.split("=");
 alert("Name="+array[0]+" "+"Value="+array[1]);
 }
 else
 {
 alert("Cookie not available");
 }
 }
 </script>
</body>
</html>
```

## 4.11.jQuery Syntax; jQuery Selectors (Element, Id, Class), jQuery Events (Mouse, Keyboard, Form, Document/Window) and jQuery Effects (Hide/Show, Fade, Slide, Animate, Stop, Callback, Chaining )

### What is jQuery?

- jQuery is a lightweight Javascript library which is blazing fast and concise. This library was created by John Resig in 2006 and
- jQuery has been designed to simplify HTML DOM tree traversal and manipulation, as well as event handling, CSS animation, and Ajax.
- jQuery can be used to find a particular HTML element in the HTML document with a certain ID, class or attribute and later we can use jQuery to change one or more of attributes of the same element like color, visibility etc. jQuery can also be used to make a webpage interactive by responding to an event like a mouse click.
- jQuery is free, open-source software which comes under the permissive MIT License. As of Apr 2021, jQuery is used by 77.8% of the top 10 million most popular websites.

jQuery has been developed with the following principles:

- **Separation of JavaScript and HTML**, which encourages developers to completely separate JavaScript code from HTML markup.
- **Brevity and clarity** promotes features like **chainable** functions and shorthand function names.
- **Eliminates of cross-browser incompatibilities**, so developers does not need to worry about browser compatibility while writing code using jQuery library.
- **Extensibility**, which means new events, elements, and methods can be easily added in jQuery library and then reused as a plugin.

# How to download and run JQuery

Step-1 Go to jQuery web site i.e

<https://jquery.com/download>

Step-2 Select Compressed

[compressed, production jQuery 3.6.3](#)

Step-3 save it in your folder of JQuery

## Class Demo

### Downloading jQuery

Compressed and uncompressed copies of jQuery files are available. The uncompressed file is best used during development or debugging; the compressed file saves bandwidth and improves performance in production. You can also download a sourcemap file for use when debugging with a compressed file. The map file is not required for users to run jQuery, it just improves the developer's debugger experience. As of jQuery 1.11.0/2.1.0 the `//# sourceMappingURL` comment is not included in the compressed file.

















